

DD: SUB-WF-RESTRICT-01 — Subscription Restriction

Developer implementation guide:

DD_SUB-WF-RESTRICT-01-Subscription_Restriction-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract

DD_SUB-WF-RESTRICT-01-Subscription_Restriction-v1.0.md.

Verdict: ■ New | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.0 Operation that applies or removes a partial-service restriction on an ACTIVE subscription. Owns: the 2-intent taxonomy (ADD / REMOVE), role gating, dunning-marker protection, per-operator config catalog, the trigger REST endpoints, the rich domain Kafka events. Composes the framework primitives from **DD_SUB-WF-FRAMEWORK-01** (parent) +

DD_SUB-WF-FRAMEWORK-01-WORKERS (catalog). **Distinct characteristic:** RESTRICT is the ONE subscription operation that does NOT mutate status_code. The subscription stays ACTIVE; only the active_restrictions[] JSONB array on the subscription is mutated. No PENDING_* transient state is used. This makes RESTRICT non-state-affecting and concurrent-safe with other operations per framework R-FW-1. **Companion DDs** (per-operator BPMN satellites): - **DD_SUB-WF-RESTRICT-01-FLOW-WIK** — KE WIK reference BPMN (shipped v1.0) - **DD_SUB-WF-RESTRICT-01-FLOW-WUG / -WTZ / -YASSN** — pending operator deep dives

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/04_subscription_workflows/DD_SUB-WF-RESTRICT-01-Subscription_Restriction-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	rules.subscription.restrict.<operator>

4. Runtime Contracts To Implement

4.1 APIs

- DELETE /api/subscriptions/{subscription_id}/restrictions/{code}
- GET /api/subscriptions/{subscription_id}/restrictions
- POST /api/subscriptions/{subscription_id}/restrictions

4.2 Tables

- subscription_restrict_config

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- bil01-emit-intent
- drools-decide
- ful-call-restrict
- kafka-emit
- notification-send
- put-active-restrictions
- restriction-approval-decision
- sub-lm-compute-cycle-window
- sub-lm-put-active-restrictions
- subscription-operation-finalize

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- SubscriptionRestrictionAddRejected
- SubscriptionRestrictionAdded
- SubscriptionRestrictionRemoveRejected
- SubscriptionRestrictionRemoved

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.

7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.0

Operation that applies or removes a partial-service restriction on an ACTIVE subscription. Owns: the 2-intent taxonomy (ADD / REMOVE), role gating, dunning-marker protection, per-operator config catalog, the trigger REST endpoints, the rich domain Kafka events. Composes the framework primitives from **DD_SUB-WF-FRAMEWORK-01** (parent) + **DD_SUB-WF-FRAMEWORK-01-WORKERS** (catalog).

Distinct characteristic: RESTRICT is the ONE subscription operation that does NOT mutate status_code. The subscription stays ACTIVE; only the active_restrictions[] JSONB array on the subscription is mutated. No PENDING_* transient state is used. This makes RESTRICT non-state-affecting and concurrent-safe with other operations per framework R-FW-1.

Companion DDs (per-operator BPMN satellites):

- **DD_SUB-WF-RESTRICT-01-FLOW-WIK** — KE WIK reference BPMN (shipped v1.0)
- **DD_SUB-WF-RESTRICT-01-FLOW-WUG / -WTZ / -YASSN** — pending operator deep dives

Glossary

Term	Meaning
Restriction	A partial-service constraint applied to an ACTIVE subscription without changing its status. Catalog defined in SUB-LM-01's subscription_restriction table (codes like OUTGOING_VOICE_BARRED, DATA_THROTTLED_LOW, BLOCK_PREMIUM_CONTENT).
active_restrictions[]	JSONB array column on subscription (owned by SUB-LM-01) listing the currently-active restriction codes for that subscription plus per-restriction metadata (added_at, added_by, dunning_marker, reason).
Intent	The verb: ADD (apply a new restriction) or REMOVE (lift an existing one).
fulfillment_action	The opaque action code from the subscription_restriction catalog row (e.g., AAA_RESTRICT_OUTGOING_VOICE). FUL-04 translates this into vendor commands.

Term	Meaning
Dunning marker	A flag in the restriction's metadata indicating the restriction was applied by BIL-04 dunning (SUB-WF-SUSPEND-NP-01 or a future dunning-driven RESTRICT). Human-actor REMOVE attempts on dunning-marked restrictions are rejected; they're cleared only by BIL-04's resume-after-payment flow.
Activation trigger	The channel that requested the restriction operation. One of BACKOFFICE_OPERATOR / CALL_CENTER / CUSTOMER_SELF_SERVICE / DUNNING_DRIVEN. Drives role gating + dunning-marker assignment.

1. What this is

The restriction operation. SUB-WF-RESTRICT-01 is one of the 10 operation kinds in the framework, and the only one that does not mutate `status_code`. Two intents (ADD and REMOVE) share the same workflow shape; intent is carried as a process variable.

ADD path: validate subscription is ACTIVE → look up restriction catalog row → check restriction not already active → check role gating + customer-self-service eligibility → call billing intent (RESTRICTION_ADD_INTENT — usually no fee but operator may declare BillableEvents) → call FUL-04 with intent=APPLY_RESTRICTION → on success, append to `active_restrictions[]` via SUB-LM-01 put-active-restrictions → emit SubscriptionRestrictionAdded.

REMOVE path: validate subscription is ACTIVE → check restriction is currently active → check role gating + dunning-marker → call billing intent (RESTRICTION_REMOVE_INTENT — may carry reactivation fee for some restrictions) → call FUL-04 with intent=LIFT_RESTRICTION (with `remainingActiveRestrictions[]` for adapter logic) → on success, remove from `active_restrictions[]` → emit SubscriptionRestrictionRemoved.

What this DD owns

1. **subscription_restrict_config table** — per-operator config: customer-self-service enablement, dunning-marker enforcement on removal, default reason taxonomy.
2. **Intent taxonomy** — 2 intents with role-gating rules.
3. **Trigger taxonomy** — 4 channel triggers.
4. **Dunning marker rules** — when set, by whom, when human-actor REMOVE is rejected.
5. **Trigger REST endpoints** — POST `/api/subscriptions/{subscription_id}/restrictions` (ADD) + DELETE `/api/subscriptions/{subscription_id}/restrictions/{code}` (REMOVE).

API convention note: most subscription operations use verb endpoints such as `/activate` or `/pause` because they mutate the subscription status through a single operation verb. RESTRICT intentionally uses a sub-resource collection because restrictions can be listed, added, and removed independently while the subscription remains ACTIVE. This is the approved exception to the MACD verb-suffix convention.

6. **Domain Kafka events** — SubscriptionRestrictionAdded + SubscriptionRestrictionRemoved + SubscriptionRestrictionAddRejected + SubscriptionRestrictionRemoveRejected.
7. **Restriction workflow rules** — 15 rules in §4.

What this DD does NOT own

- **BPMN files** — owned by per-operator satellites.
- **Worker implementations** — catalogued in DD_SUB-WF-FRAMEWORK-01-WORKERS.
- **The subscription_restriction catalog itself** — owned by SUB-LM-01.
- **The active_restrictions[] JSONB column** — owned by SUB-LM-01.
- **The put-active-restrictions API** — owned by SUB-LM-01.
- **Network-side restriction enforcement** — owned by FUL-04 (translates fulfillment_action codes to vendor commands).
- **Restriction-related billing fees** — owned by BIL-CFG-01 + BIL-01 (BillableEvents filter on restriction code).
- **Dunning ladder orchestration** — owned by BIL-04.

Regional flexibility

Each operator's RESTRICT flow is its own BPMN file. KE WIK's BPMN composes ~8 workers. Operators with simpler restriction taxonomies may compose fewer; operators requiring supervisor approval for sensitive restrictions add a user task.

Per-operator levers:

Lever	Examples for RESTRICT
BPMN file	KE: no supervisor approval needed; restriction takes effect immediately. Other operators may add restriction-approval-decision user task for high-impact restrictions.
Drools package rules.subscription.restrict.<operator>	KE: customer-self-service eligible restrictions list, dunning-marker enforcement on REMOVE, reactivation-fee triggers.
subscription_restrict_config row	KE: customer_self_service_enabled=true for non-dunning restrictions; dunning_marker_strict=true.
FOUNDATION_AUTH candidate groups	Operator-specific resolution.

2. Tables overview

This DD owns one table: subscription_restrict_config (per-operator config).

Cross-operation audit anchor: subscription_operation (framework-owned).

The subscription_restriction catalog (defining codes like OUTGOING_VOICE_BARRED with their fulfillment_action) lives in SUB-LM-01. The active_restrictions[] JSONB array is a column on subscription (also SUB-LM-01). Workers write the array via sub-lm-put-active-restrictions.

3. Schemas

3.1 subscription_restrict_config

One row per operator.

```
CREATE TABLE subscription_restrict_config (
  operator_code          TEXT PRIMARY KEY,
  customer_self_service_enabled  BOOLEAN NOT NULL DEFAULT FALSE,
  dunning_marker_strict    BOOLEAN NOT NULL DEFAULT TRUE,
  restriction_approval_required_for_codes  JSONB NOT NULL DEFAULT '[]',
  updated_at              TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_by              TEXT NOT NULL
);
```

Column	Notes
customer_self_service_enabled	When true, customers may add/remove restrictions from their portal — but only for catalog entries with customer_self_service_eligible = true. When false, the CUSTOMER_SELF_SERVICE role is not accepted on this operator's trigger endpoints regardless of catalog flags.
dunning_marker_strict	When true (default), human-actor REMOVE on dunning-marked restrictions is rejected with 403 SYSTEM_MANAGED_RESTRICTION (BIL-04 must lift them). When false, admin actors with SUBSCRIPTION_ADMIN role may override.
restriction_approval_required_for_codes	JSONB array of restriction code strings (from SUB-LM-01 catalog) that require a restriction-approval-decision user task before commit. Empty by default; operators populate as needed. Example: ["FULL_SERVICE_SUSPENSION_PUNITIVE"].

Sample data:

operator_code	customer_self_service_enabled	dunning_marker_strict	restriction_approval_required_for_codes
WIK	true	true	[]
WUG	false	true	[]
WTZ	false	true	[]
YASSN	false	true	[]

4. Rules

4.1 State precondition + array mutation

Rule	Description
R-SUB-WF-RESTRICT-01-S-1	Both ADD and REMOVE require the subscription's <code>status_code = ACTIVE</code> . Restrictions cannot be added/removed on <code>SUSPENDED</code> , <code>PENDING_*</code> , <code>TERMINATED</code> , <code>RETIRED</code> , or <code>CREATED</code> . Returns 409 <code>INVALID_STATE_FOR_RESTRICTION</code> with the current status.
R-SUB-WF-RESTRICT-01-S-2	The subscription's <code>status_code</code> does NOT change during restriction operations. Only the <code>active_restrictions[]</code> JSONB array is mutated. No <code>PENDING_*</code> transient state is used.
R-SUB-WF-RESTRICT-01-S-3	Per framework R-FW-1, <code>RESTRICT</code> is non-exclusive: it MAY run concurrently with any other in-flight operation on the same subscription (since neither side mutates <code>status_code</code>). Two <code>RESTRICT</code> operations on the same subscription are serialized via row-level lock on the subscription record during the <code>put-active-restrictions</code> call.

4.2 Intent rules

Rule	Description
R-SUB-WF-RESTRICT-01-I-1	ADD: if the restriction code is already in <code>active_restrictions[]</code> , returns 409 <code>ALREADY_RESTRICTED</code> with the existing entry's metadata.
R-SUB-WF-RESTRICT-01-I-2	REMOVE: if the restriction code is NOT in <code>active_restrictions[]</code> , returns 409 <code>RESTRICTION_NOT_APPLIED</code> .
R-SUB-WF-RESTRICT-01-I-3	Multiple distinct restrictions may be active simultaneously on the same subscription. Each is added or removed independently via separate workflow runs.
R-SUB-WF-RESTRICT-01-I-4	The optional <code>notes</code> field on both ADD and REMOVE requests captures the operator's justification. Audit-logged in <code>subscription_operation.failure_reason_detail</code> (success path) and <code>active_restrictions[].add_notes / release_notes</code> .

4.3 Catalog validation

Rule	Description
R-SUB-WF-RESTRICT-01-C-1	The restriction code must reference an active row in SUB-LM-01's <code>subscription_restriction</code> catalog (where <code>is_active = true</code>). Unknown codes return 400 <code>UNKNOWN_RESTRICTION_CODE</code> . Inactive codes return 400 <code>RESTRICTION_CODE_INACTIVE</code> .
R-SUB-WF-RESTRICT-01-C-2	The catalog row supplies <code>fulfillment_action</code> (FUL-04 input), <code>customer_self_service_eligible</code> , <code>admin_only</code> , <code>is_active</code> . The workflow snapshots these into the workflow's process variables at validation time.
R-SUB-WF-RESTRICT-01-C-3	A <code>CUSTOMER_SELF_SERVICE</code> -role request requires (a) <code>subscription_restrict_config.customer_self_service_enabled = true</code> for the operator AND (b) the catalog row's <code>customer_self_service_eligible = true</code> . Mismatch → 403 <code>CUSTOMER_SELF_SERVICE_NOT_ALLOWED</code> .
R-SUB-WF-RESTRICT-01-C-4	A request against a catalog row with <code>admin_only = true</code> requires <code>SUBSCRIPTION_ADMIN</code> role. Insufficient role → 403 <code>ADMIN_ONLY_RESTRICTION</code> .

4.4 Role gating per trigger

Rule	Description
R-SUB-WF-RESTRICT-01-RG-1	Role-trigger gating: <code>BACKOFFICE_OPERATOR</code> requires <code>SUBSCRIPTION_OPERATOR</code> or <code>SUBSCRIPTION_ADMIN</code> . <code>CALL_CENTER</code> requires <code>CALL_CENTER</code> . <code>CUSTOMER_SELF_SERVICE</code> requires <code>CUSTOMER_SELF_SERVICE</code> plus the catalog + config checks per R-C-3. <code>DUNNING_DRIVEN</code> requires <code>BILLING_INTERNAL</code> .
R-SUB-WF-RESTRICT-01-RG-2	The framework's <code>drools-decide</code> worker is invoked with <code>rules.subscription.restrict.<operator></code> and the request facts; the operator-scoped package enforces role-gating + may add operator-specific rules.

4.5 Dunning marker

Rule	Description
R-SUB-WF-RESTRICT-01-DM-1	When <code>activationTrigger = DUNNING_DRIVEN</code> , the added restriction's metadata includes <code>dunning_marker = true</code> and <code>dunning_reference</code> (BIL-04 dunning wave or step ID). Set automatically by the workflow; cannot be overridden by caller.
R-SUB-WF-RESTRICT-01-DM-2	When <code>dunning_marker_strict = true</code> (operator config), a human-actor REMOVE (any non-BILLING_INTERNAL role) on a dunning-marked restriction returns 403 SYSTEM_MANAGED_RESTRICTION with guidance: "this restriction was applied by automated dunning; settle the outstanding balance to remove it".
R-SUB-WF-RESTRICT-01-DM-3	When <code>dunning_marker_strict = false</code> , a SUBSCRIPTION_ADMIN role MAY override dunning-marker protection. The override requires explicit <code>dunning_override = true</code> in the REMOVE request. Audit-logged in the SubscriptionRestrictionRemoved event with <code>override_actor</code> and <code>override_reason</code> .
R-SUB-WF-RESTRICT-01-DM-4	A BILLING_INTERNAL actor (BIL-04 resume-after-payment) may always remove a dunning-marked restriction.

5. REST API

5.1 Add a restriction

POST /api/subscriptions/{subscription_id}/restrictions HTTP/1.1
 Authorization: Bearer <jwt>
 Content-Type: application/json
 Idempotency-Key: restrict-add-sub_01HZ_KAMAU-OUTGOING_VOICE_BARRED-dunning-L2

```
{
  "operatorCode":      "WIK",
  "restrictionCode":   "OUTGOING_VOICE_BARRED",
  "activationTrigger": "DUNNING_DRIVEN",
  "dunningReference":  "bil04-dunning-wave-2026-05-21-L2",
  "notes":             "Dunning level 2 – voice outgoing barred until invoice INV_01HZ paid",
  "initiatingActorUserId": "bil04-svc-account",
  "initiatingActorRole": "BILLING_INTERNAL",
  "initiatingWorkflowKind": "BIL_04_DUNNING",
  "initiatingWorkflowRef": "bil04-dunning-wave-2026-05-21-L2"
}
```

→ 202 Accepted

```
{
  "operationId":      "subop_01HZ_KAMAU_RESTRICT_ADD",
  "operatorCode":     "WIK",
  "subscriptionId":   "sub_01HZ_KAMAU_FIBER",
  "operationKind":    "RESTRICT",
  "intent":           "ADD",
  "restrictionCode":  "OUTGOING_VOICE_BARRED",
  "bpmnProcessKey":   "sub-restrict-wik",
  "bpmnProcessInstanceId": "pi_01HZ_KAMAU_RESTRICT_ADD",
  "currentState":     "VALIDATING",
  "startedAt":        "2026-05-21T02:00:00+03:00"
}
```

Already restricted:

→ 409 Conflict

```
{
  "code":      "ALREADY_RESTRICTED",
  "restrictionCode": "OUTGOING_VOICE_BARRED",
  "existingEntry": {
    "addedAt":      "2026-05-21T01:55:00+03:00",
    "addedByActor": "bil04-svc-account",
    "addNotes":     "Dunning level 2 prior attempt"
  }
}
```

Unknown restriction code:

→ 400 Bad Request

```
{ "code": "UNKNOWN_RESTRICTION_CODE", "restrictionCode": "BOGUS_RESTRICTION" }
```

Customer self-service not allowed:

→ 403 Forbidden

```
{
  "code":      "CUSTOMER_SELF_SERVICE_NOT_ALLOWED",
  "restrictionCode": "OUTGOING_VOICE_BARRED",
  "catalogCustomerSelfServiceEligible": false
}
```

```
}
```

5.2 Remove a restriction

```
DELETE /api/subscriptions/{subscription_id}/restrictions/{code} HTTP/1.1
Authorization: Bearer <jwt>
Content-Type: application/json
Idempotency-Key: restrict-remove-sub_01HZ_KAMAU-OUTGOING_VOICE_BARRED-bil04-resume-20260601
```

```
{
  "operatorCode":      "WIK",
  "notes":             "Customer settled invoice INV_01HZ on 2026-06-01",
  "dunningOverride":   false,
  "initiatingActorUserId": "bil04-svc-account",
  "initiatingActorRole": "BILLING_INTERNAL",
  "initiatingWorkflowKind": "BIL_04_DUNNING",
  "initiatingWorkflowRef": "bil04-resume-2026-06-01"
}
```

→ 202 Accepted

```
{
  "operationId":      "subop_01HZ_KAMAU_RESTRICT_REMOVE",
  "operatorCode":     "WIK",
  "subscriptionId":   "sub_01HZ_KAMAU_FIBER",
  "operationKind":     "RESTRICT",
  "intent":           "REMOVE",
  "restrictionCode":  "OUTGOING_VOICE_BARRED",
  "bpmnProcessKey":   "sub-restrict-wik",
  "bpmnProcessInstanceId": "pi_01HZ_KAMAU_RESTRICT_REMOVE",
  "currentState":     "VALIDATING",
  "startedAt":        "2026-06-01T10:00:00+03:00"
}
```

Restriction not applied:

```
→ 409 Conflict
{ "code": "RESTRICTION_NOT_APPLIED", "restrictionCode": "OUTGOING_VOICE_BARRED" }
```

Dunning-marked, human actor blocked:

```
→ 403 Forbidden
{
  "code":      "SYSTEM_MANAGED_RESTRICTION",
  "restrictionCode": "OUTGOING_VOICE_BARRED",
  "dunningMarker": true,
  "dunningReference": "bil04-dunning-wave-2026-05-21-L2",
  "guidance": "Settle the outstanding balance to clear this restriction. Contact billing if the balance is set"
}
```

5.3 List active restrictions on a subscription

```
GET /api/subscriptions/{subscription_id}/restrictions HTTP/1.1
```

→ 200 OK

```
{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "activeRestrictions": [
    {
      "restrictionCode": "OUTGOING_VOICE_BARRED",
      "addedAt":        "2026-05-21T02:00:00+03:00",
      "addedByActor":   "bil04-svc-account",
      "addedByRole":    "BILLING_INTERNAL",
      "addNotes":       "Dunning level 2 – voice outgoing barred",
      "dunningMarker":  true,
      "dunningReference": "bil04-dunning-wave-2026-05-21-L2"
    }
  ]
}
```

6. Kafka events emitted

Event	Topic	Fires on	Consumers
SubscriptionRestrictionAdded	sophix.subscription.subscription.restrictionadded	ADD success	NOT-01 (customer notification — content depends on dunning marker), BIL-04 (acknowledges its dunning step), CVM analytics, EM-04

Event	Topic	Fires on	Consumers
SubscriptionRestrictionRemoved	sophix.subscription.subscription.restrictionremoved	REMOVE success	NOT-01 (customer notification), BIL-04 (acknowledges resume), CVM analytics, EM-04
SubscriptionRestrictionAddRejected	sophix.subscription.subscription.restrictionaddrejected	ADD failure	NOT-01 (notify actor if backoffice), ICN-01 ops alert, EM-04
SubscriptionRestrictionRemoveRejected	sophix.subscription.subscription.restrictionremoverejected	REMOVE failure	NOT-01, ICN-01, EM-04

Sample SubscriptionRestrictionAdded:

```
{
  "eventType": "SubscriptionRestrictionAdded",
  "aggregateId": "sub_01HZ_KAMAU_FIBER",
  "timestamp": "2026-05-21T02:00:18+03:00",
  "payload": {
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HZ_KAMAU_FIBER",
    "customerId": "cust_01HZ_KAMAU",
    "operationId": "subop_01HZ_KAMAU_RESTRICT_ADD",
    "restrictionCode": "OUTGOING_VOICE_BARRED",
    "fulfillmentAction": "AAA_RESTRICT_OUTGOING_VOICE",
    "activationTrigger": "DUNNING_DRIVEN",
    "dunningMarker": true,
    "dunningReference": "bil04-dunning-wave-2026-05-21-L2",
    "addedAt": "2026-05-21T02:00:18+03:00",
    "addedByActorUserId": "bil04-svc-account",
    "addedByActorRole": "BILLING_INTERNAL",
    "previousActiveRestrictions": [],
    "newActiveRestrictions": ["OUTGOING_VOICE_BARRED"]
  }
}
```

Sample SubscriptionRestrictionRemoved with override:

```
{
  "eventType": "SubscriptionRestrictionRemoved",
  "aggregateId": "sub_01HZ_KAMAU_FIBER",
  "payload": {
    "subscriptionId": "sub_01HZ_KAMAU_FIBER",
    "restrictionCode": "OUTGOING_VOICE_BARRED",
    "fulfillmentAction": "AAA_RESTRICT_OUTGOING_VOICE",
    "removedAt": "2026-06-01T10:00:18+03:00",
    "removedByActorUserId": "bil04-svc-account",
    "removedByActorRole": "BILLING_INTERNAL",
    "dunningOverride": false,
    "newActiveRestrictions": []
  }
}
```

7. Cross-DD coordination

7.1 Upstream callers

Caller	Endpoint	Trigger value	Role
BIL-04 Dunning	POST .../restrictions + DELETE .../restrictions/{code}	DUNNING_DRIVEN	BILLING_INTERNAL
BO admin (manual restriction)	Same	BACKOFFICE_OPERATOR	SUBSCRIPTION_OPERATOR or SUBSCRIPTION_ADMIN
Call Center	Same	CALL_CENTER	CALL_CENTER
Customer self-service portal	Same	CUSTOMER_SELF_SERVICE	CUSTOMER_SELF_SERVICE (subject to config + catalog flags)

7.2 Sibling modules

Module	Direction	Contract
SUB-WF-FRAMEWORK-01 (parent)	This DD composes framework primitives	subscription_operation row created per restriction op; BPMN started

Module	Direction	Contract
SUB-WF-FRAMEWORK-01-WORKERS	This DD's BPMNs reference workers by topic	Workers used: drools-decide, sub-lm-compute-cycle-window (for billing context), bil01-emit-intent, ful-call-restrict, sub-lm-put-active-restrictions, kafka-emit, notification-send, subscription-operation-finalize
SUB-LM-01	Workers read catalog + write active_restrictions[]	Catalog read for restriction code lookup; put-active-restrictions for array mutation
FUL-04	ful-call-restrict worker	APPLY_RESTRICTION (ADD) or LIFT_RESTRICTION (REMOVE)
BIL-04 Dunning	Calls trigger endpoints; consumes events	Drives dunning-driven restrictions at L2; consumes SubscriptionRestrictionAdded to acknowledge step
BIL-01	bil01-emit-intent worker	RESTRICTION_ADD_INTENT or RESTRICTION_REMOVE_INTENT (may carry reactivation fees)
NOT-01	notification-send worker	Customer notification when restriction applied/removed
CVM / EM-04	Consume Kafka events	Analytics + reporting

7.3 Build order

This DD deploys after:

1. SUB-WF-FRAMEWORK-01 (parent + WORKERS) v1.0
2. SUB-LM-01 v1.0
3. FUL-04 v1.0
4. BIL-01
5. At least one FLOW satellite (FLOW-WIK v1.0)

7.4 Stub debt + open coordination items

- **FLOW satellites for WUG / WTZ / YASSN** — pending operator deep dives.
- **BIL-04 Dunning ladder** — BIL-04 needs to call this DD's trigger endpoint at dunning step L2. The integration is captured in BIL-04 v1.4 (already shipped) referencing this DD.
- **BIL-CFG-01 RESTRICTION_ADD_INTENT / RESTRICTION_REMOVE_INTENT BillableEvent definitions** — to be added in BIL-CFG-01 v1.0 promotion (operator-tuned; KE v1.0 has no restriction fees; reactivation fee future).

B — Current TZ

Status: To be filled in after codebase cartography review.

Workshop materials note that legacy dunning uses an AAA restrict/restore pattern at the suspension step (Billing → AAA: restrict / restore). Whether the legacy codebase has a structured restriction catalog or treats restrictions as ad-hoc AAA rules, and what the legacy mapping between dunning steps and restriction taxonomy looks like, require source-code review to document. Until that cartography lands, this DD does not assert specific legacy structural claims.